



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2513 - TECNOLOGÍAS Y APLICACIONES WEB

Entrega 2

Tecnologías y Aplicaciones Web
2º semestre 2026 - Profesor Hernán Cabrera

Fecha de Entrega: Domingo 27 de Septiembre a las 23:59 hrs.

Introducción

En la primera entrega definieron los cimientos conceptuales, las reglas del juego, los diagramas estructurales y los primeros mockups visuales.

El objetivo de esta **Entrega 2** es dar el salto al código e implementar la **capa cliente (Frontend)** de su aplicación web utilizando el framework **React**, junto con formalizar la **especificación del protocolo de comunicación JSON** que gobernará la interacción entre el navegador y el futuro servidor.

En esta etapa, el Frontend cobrará vida mediante una aplicación de página única (*Single Page Application* o SPA) navegable, modular y visualmente atractiva, alimentada provisionalmente por **datos mock (simulados)** que sigan la estructura exacta del protocolo definido.

Indicaciones Generales

Para esta entrega deberán transformar sus mockups y reglas en un conjunto de componentes en `.jsx` y hojas de estilo en `CSS`, cumpliendo con los siguientes requerimientos principales:

- Especificación Formal del Protocolo JSON:** Definir y documentar los contratos de datos (*Requests* y *Responses*) de todas las acciones del juego.
- Desarrollo Frontend en React:** Implementación modular de todas las vistas comprometidas en componentes independientes.
- Navegación SPA:** Configuración fluida entre páginas mediante **React Router** con una barra de navegación siempre accesible.
- Estilos y Maquetación:** Uso de `CSS` modular, `Flexbox` y `CSS Grid` sin recurrir a estilos *inline* ni frameworks externos (`Bootstrap`, `Tailwind`, etc.).
- Datos Mock Consistentes:** La interfaz debe consumir datos simulados en memoria o archivos `JSON` locales para poblar dinámicamente las vistas del juego y de los usuarios.

- f. **Gestión del Proyecto:** Mantenimiento activo del tablero Kanban y registro de decisiones en el repositorio.

Requerimientos Detallados

1. Especificación del Protocolo de Comunicación JSON

A partir del modelo Entidad-Relación y el diagrama de clases de la Entrega 1, deberán definir detalladamente la estructura de los objetos JSON que se transferirán entre cliente y servidor.

Deberán documentar, en formato de tabla o especificación técnica en el `README.md` (o un documento en `/docs`):

- **Estructura de Requests:** Qué datos envía el cliente al servidor al realizar una acción (e.g., registro/login, crear partida, realizar un movimiento, comprar recursos, pasar turno, abandonar).
- **Estructura de Responses:** Qué datos devuelve el servidor tanto en casos de éxito (código `2xx` y nuevo estado del juego/tablero) como en casos de jugadas inválidas o errores (códigos `4xx/5xx` con mensaje explicativo).
- **Objeto Estado del Juego:** Estructura del JSON que representa el estado completo de una partida activa (identificador de turno, jugadores conectados, posiciones en el tablero, puntajes, recursos).

Nota: Este protocolo es preliminar y podrá recibir ajustes justificados en las entregas posteriores de integración backend.

2. Vistas Principales en React (.jsx)

El cliente web debe contar como mínimo con las siguientes vistas desarrolladas en componentes reutilizables:

1. **Página de Inicio (*Landing Page*):** Punto de entrada con información general del proyecto, motivación, botones claros para registrarse o iniciar sesión, y un acceso directo a las partidas.
2. **Página de Instrucciones y Reglas:** Vista estática que explique claramente cómo se juega, mecánicas de turnos, condiciones de victoria/derrota y significado de los elementos del tablero.
3. **Página de Nosotros (*About Us*):** Información del equipo de desarrollo, roles y descripción del proyecto.
4. **Página de Registro e Inicio de Sesión (*Login / Sign Up*):** Formularios interactivos con validación visual en el cliente (campos obligatorios, formato de correo, coincidencia de contraseñas y selección de rol inicial).
5. **Pantalla de Juego / Tablero:** Vista principal de la partida. Debe ser de diseño modular (e.g., componente Tablero, componente Casilla/Ficha, componente Panel de Control del Jugador, componente Historial/Bitácora de Acciones).

3. Navegación con React Router

La aplicación no debe recargar la página completa al cambiar de vista. Deben utilizar **React Router** para manejar las rutas del lado del cliente.

- Debe existir una barra de navegación (`<nav>`) global y accesible en todo momento.
- Se debe simular el cambio de estado de sesión (por ejemplo, alternar visualmente entre “Iniciar Sesión” y el perfil del usuario activo una vez simulado el ingreso).

4. Estilos, Layouts y Responsividad

- **Prohibido el uso de estilos *inline*** (`style={{...}}`): Toda la presentación debe manejarse en archivos `.css` separados por componente o módulos de CSS.
- **Prohibido el uso de frameworks CSS** (Bootstrap, Tailwind, Bulma, Chakra UI, etc.). Se evaluará el dominio de CSS nativo.
- Deben aplicar el reseteo del modelo de caja (`box-sizing: border-box`) y utilizar modelos de maquetación modernos como **Flexbox** (para barras de navegación, botones y alineaciones internas) y **CSS Grid** (para la estructura del tablero y grillas de contenido).
- La interfaz debe ser responsiva y adaptarse de manera fluida a distintos tamaños de pantalla.

5. Uso de Datos Mock y Estado Local

Dado que el servidor backend real se implementará en entregas futuras, el Frontend debe demostrar su dinamismo mediante el manejo de estado en React (`useState`, `useEffect`, etc.):

- Crear archivos de datos mock (en formato `.json` o `.js`) que contengan listas de usuarios, partidas simuladas y configuraciones de tablero que sigan el protocolo JSON definido.
- Las vistas deben renderizar listas y elementos a partir de estos datos simulados mediante funciones como `.map()`.
- En la pantalla de juego, se deben poder realizar interacciones básicas visuales (e.g., seleccionar una casilla, simular el avance de un turno o cambiar valores de recursos en la interfaz).

6. Despliegue del Frontend (PaaS)

Se recomienda fuertemente realizar el despliegue del Frontend en una plataforma PaaS como **Netlify** o **Vercel**, dejando el sitio accesible públicamente a través de una URL de Internet. La URL debe incluirse en el `README.md`.

7. Gestión del Proyecto y Documentación

- **README.md:** Debe incluir las instrucciones paso a paso para clonar, instalar dependencias (`npm install`) y levantar la aplicación en entorno local (`npm run dev` o `npm start`), además del enlace a la versión desplegada.

- **Tablero Kanban:** Debe reflejar las tareas realizadas durante la Entrega 2, con asignación de responsables, fechas límite y estados de avance (Por hacer, En progreso, Finalizado). Se debe adjuntar el enlace o captura del tablero en la carpeta `/docs`.

Condiciones y Restricciones

- La aplicación debe ser desarrollada en **React** con **JavaScript** estándar. No se permite el uso de lenguajes que compilen a JS como TypeScript o CoffeeScript.
- Uso de **HTML semántico** obligatorio (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`). No estructurar todo el sitio exclusivamente con etiquetas `<div>`.
- **Uso obligatorio de GitHub:** La entrega se evaluará directamente sobre el repositorio oficial de Frontend asignado a su grupo. Todos los integrantes deben registrar commits significativos y trabajo colaborativo mediante *Pull Requests*.
- **No se aceptarán entregas por correo electrónico ni fuera del plazo establecido.**

Sobre la Calificación

La calificación busca evaluar la calidad del diseño de la interfaz, la correcta modularización en React, la fidelidad al protocolo propuesto y la experiencia de usuario general.

Criterios de Evaluación

Criterio	Esencial	Intermedio	Completo	Pje.
1. Protocolo JSON Cliente-Servidor	Definición incompleta o solo de 1 acción	Estructura de jugadas y estados definida	Protocolo formal, detallado con casos de éxito, error y estructura de partida	15
2. Vistas principales en React	Faltan vistas solicitadas	Vistas creadas pero con contenido estático plano	Todas las vistas requeridas creadas en componentes modulares y legibles	15
3. Pantalla de Juego / Tablero	Tablero sin división modular	Tablero visible con interacción rígida	Diseño modular (tablero, controles, estado, bitácora) con componentes reutilizables	15
4. Navegación con React Router	Enlaces HTML tradicionales con recarga	Rutas creadas pero con inconsistencias	SPA funcional, navegación fluida sin recargas y barra de navegación siempre visible	10
5. Estilos, Layout y Box Model	Hay estilos <i>inline</i> o mal estructurados	Estilos en CSS pero con maquetación rígida	CSS modular por componente, uso de Flexbox/Grid, sin estilos <i>inline</i>	10
6. Consumo de Datos Mock	Datos hardcodeados en el JSX	Datos mock parciales en componentes	Consumo dinámico de datos mock coherentes con el protocolo JSON	10
7. Formularios e Interacción básica	Formularios sin validación visual	Campos funcionales pero sin control de estado	Validaciones visuales en login/-registro y simulación de cambio de sesión	8
8. Responsividad y UX/UI	Rota en pantallas móviles o desktop	Adaptable pero con desbordes menores	Diseño atractivo, intuitivo, responsivo y con jerarquía visual clara	7
9. Documentación y README	README incompleto sin comandos de inicio	Instrucciones básicas de ejecución	README claro con comandos, decisiones técnicas, supuestos y link a deploy	5
10. Kanban y Gestión Ágil	Tablero sin actualizar o sin responsables	Tareas divididas por columnas	Tablero activo, trazabilidad de tareas, asignación equitativa y avance real	5

Puntaje Total: 100 puntos

Requisitos Mínimos para Nota 4.0

Para optar a una calificación igual o superior a 4.0, el grupo debe cumplir obligatoriamente en nivel **Completo** con los siguientes ítems:

1. Especificación del Protocolo JSON Cliente-Servidor.
2. Vistas principales desarrolladas en componentes React.
3. Pantalla de juego y tablero modular.
4. Navegación fluida con React Router y barra de navegación.
5. Separación de estilos en CSS modular (sin estilos *inline*) y uso de Flexbox/Grid.
6. Consumo consistente de Datos Mock.

Política de Referencia y Uso de Inteligencia Artificial

En el desarrollo del proyecto grupal, los estudiantes podrán utilizar material proporcionado por el equipo docente, recursos disponibles en internet y herramientas basadas en Inteligencia Artificial (por ejemplo, ChatGPT, Claude, GitHub Copilot, entre otras). Sin embargo, todas las fuentes utilizadas deben ser debidamente referenciadas y el equipo debe demostrar un **aporte propio significativo** en la solución implementada.

Si se utiliza Inteligencia Artificial, se deberá indicar de forma explícita, transparente y detallada en una sección dedicada del archivo README.md de la entrega (o en un documento anexo dentro de la carpeta /docs), incluyendo:

- **Alcance y contexto:** Indicar con precisión qué partes del trabajo, componentes, estilos o funciones fueron apoyadas o generadas mediante IA.
- **Registro de interacción:** Adjuntar los *prompts* utilizados, los fragmentos (*snippets*) de código entregados por la herramienta y los enlaces públicos a las conversaciones o **capturas de pantalla legibles de los chats**.
- **Aporte y análisis crítico:** Explicar qué modificaciones, adaptaciones, correcciones e integraciones realizó el equipo sobre dicho contenido para incorporarlo al proyecto.

Régimen de Penalizaciones y Descuentos:

- En caso de que el aporte de los integrantes no sea considerado sustancial (por ejemplo, si la mayor parte del trabajo corresponde a código generado automáticamente sin comprensión ni personalización por parte del grupo), la nota podrá ser rebajada según el criterio del equipo docente.
- **Uso no referenciado o referenciado incorrectamente:** El uso de herramientas de IA sin su debida declaración, o con un reporte incompleto (e.g., no adjuntar capturas, prompts o no transparentar el alcance real de su uso), constituye una falta grave a esta política y se aplicará un **descuento directo de entre 0 a 100 puntos sobre la calificación final de la entrega**, dependiendo de la magnitud y gravedad de la omisión detectada. El uso no declarado de IA podrá derivar adicionalmente en una calificación final de nota 1,1, además de las medidas disciplinarias que determine el equipo docente.